DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 5:
Shared Memory Programming with OpenMP

Parallel regions

Example:

```
1       double result = 0;
2       #pragma omp parallel reduction(+:result)
3         result = run_mc(trials) / omp_get_num_threads();
4       printf("Final result: %f\n", result);
```

Example: Numerical Integration

- Use a parallel construct: `#pragma omp parallel`.
- The challenge is to:
  - divide loop iterations between threads (use the thread ID and the number of threads).
  - Create an accumulator for each thread to hold partial sums that you can later combine to generate the global sum.

In addition to a parallel construct, you will need the runtime library routines:

```
1 int omp_get_num_threads();
2 int omp_get_thread_num();
3 double omp_get_wtime();
4 void omp_set_num_threads();
```

An environment variable to request *N* threads

```
$export OMP_NUM_THREADS=N
```

Example: Numerical Integration

- Go back to your parallel pi program and explore how well it scales with (1) the number of threads and (2) the number of steps in the integration.
- Can you explain your performance with Amdahl's law? If not what else might be going on?
- Poor scaling: if you promote scalars to an array to support creation of an SPMD program, the array elements are contiguous in memory and hence share cache lines ... results in poor scalability.

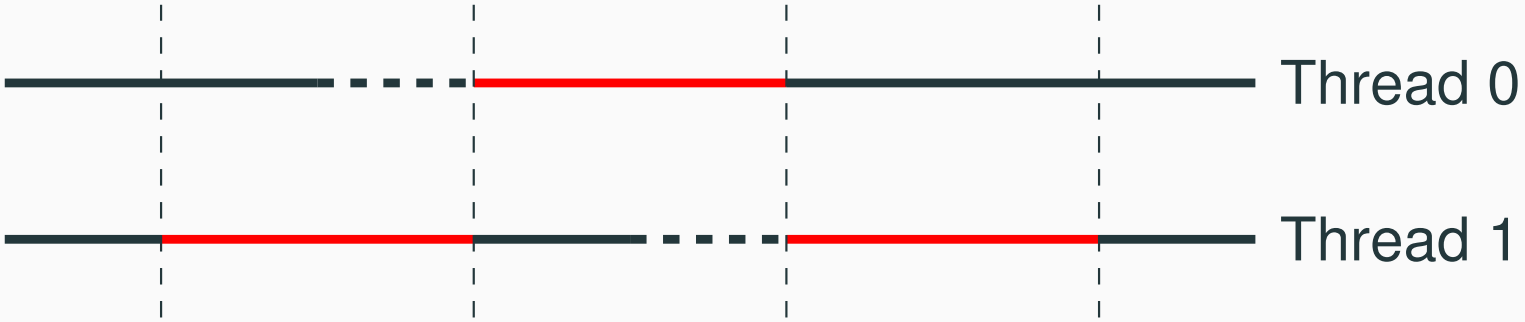
OpenMP synchronization

Synchronization is used to impose order constraints and to protect access to shared data.
High-level synchronization:

- `critical`: Critical sections
- `atomic`: Atomic update
- `barrier`: Barrier
- `ordered`: Ordered access

Critical sections

Mutual exclusion: Only one thread at a time can enter a critical region:



- Automatically lock/unlock at ends of *critical section*
- Automatically memory flushes for consistency

Critical sections

```
1
2  float res;
3
4  #pragma omp parallel
5  {
6    float B;
7    int i, id, nthrds;
8
9    id = omp_get_thread_num();
10   nthrds = omp_get_num_threads();
11
12   for(i = id; i < niters; i += nthrds){
13       B = big_job(i);
14 #pragma omp critical
15       res += consume (B); \\Threads wait their turn
16   }
17 }
```

Atomic updates

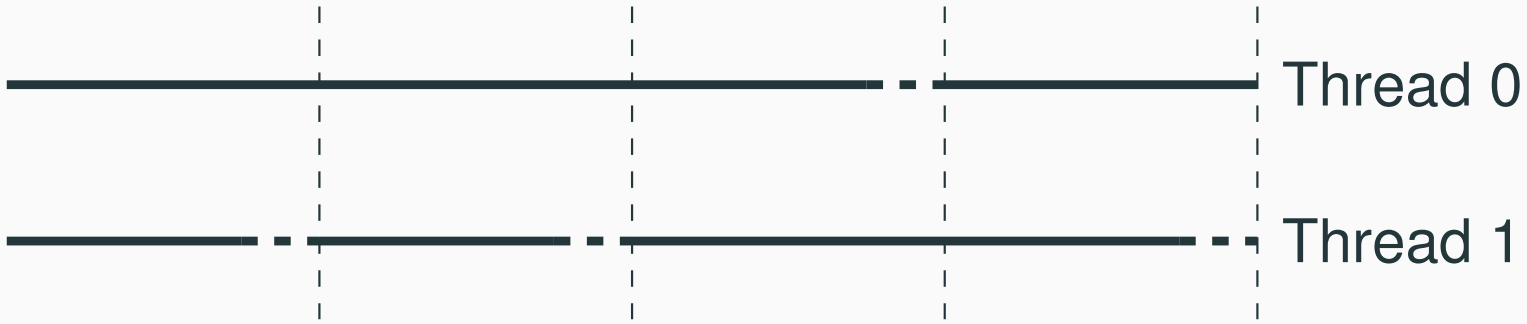
Atomic provides mutual exclusion but only applies to the update of a memory location (the update of X in the following example)

```
1 #pragma omp parallel
2 {
3   ...
4   double my_piece = foo();
5   #pragma omp atomic
6   x += my_piece;
7 }
```

Only simple ops: increment/decrement or `x += expr`. In essence, this directive provides a mini-CRITICAL section.

Barriers

Barrier: a point in a program all threads much reach before any threads are allowed to proceed.




```
#pragma omp parallel
for (i = 0; i < nsteps; ++i) {
  do_stuff
#pragma omp barrier
\\threads wait until all threads hit the barrier
}
```

Back to the π calculation

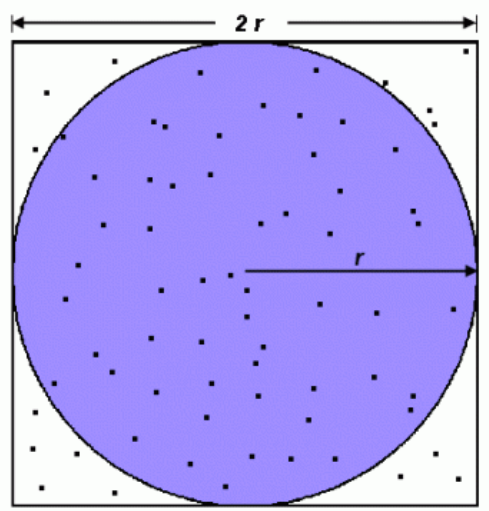
Numerical Integration exercise:

- Use an array to create space for each thread to store its partial sum.
  - If array elements happen to share a cache line, this leads to false sharing.
  - Non-shared data in the same cache line so each update invalidates the cache line ... in essence “sloshing independent data” back and forth between threads.
- Modify your “pi program” to avoid false sharing due to the sum array.

Calculating π using Monte Carlo method

Another problem that's easy to parallelize:

- All point calculations are independent; no data dependencies
- Work can be evenly divided; no load balance concerns
- No need for communication or synchronization between tasks
  - Divide the loop into equal portions that can be executed by the pool of tasks
  - Each task independently performs its work
  - One task acts as the master to collect results and compute the value of PI



More OpenMP

Next:

- Parallel loop constructs
- Task-based parallelism
- Other parallel work divisions

Summary

Programming shared memory machines

- May allocate data in large shared region without too many worries about where
- For performance tuning, watch sharing
- Avoid races or use machine-specific fences carefully

Last time

- Environmental inquiries with `omp_get_*` functions
- Creating parallel regions with `#pragma omp parallel`
- Annotations for variables (shared, private, reduction)
- Synchronization via critical sections, atomic ops, barriers
- Today: Work sharing, tasks, and some examples

Work sharing

Work sharing constructs split work across a team

- Parallel `for`: split by loop iterations
- `sections`: non-iterative tasks
- `single`: only one thread executes (synchronized)
- `master`: master executes, others skip (no sync)

Parallel iteration

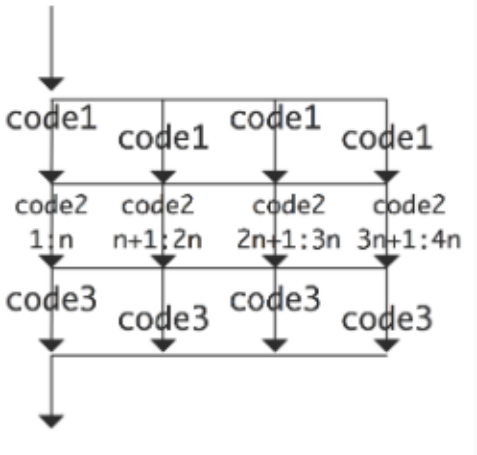
Idea: Map **independent** iterations onto different threads

```
#pragma omp parallel for
for (int i = 0; i < 4*n; ++i)
    a[i] += b[i];

#pragma omp parallel
{
    // Stuff can go here... code(1)

    #pragma omp for
    for (int i = 0; i < 4*n; ++i)
        a[i] += b[i]; // code(2)

   // code(3)
}
```



Implicit barrier at end of loop (unless `nowait` clause)

Parallel iteration

The iteration can also go across a higher-dim index set

```
1   #pragma omp parallel for collapse(2)
2   for (int i = 0; i < N; ++i)
3     for (int j = 0; j < M; ++j)
4       a[i*M+j] = foo(i,j);
```

The collapse clause is used to convert a prefect nested loop into a single loop then parallelize it.

Restrictions

- `for` loop must be in “canonical form”
  - Loop var is an integer, pointer, random access iterator (C++)
  - Test compares loop var to loop-invariant expression
  - Increment or decrement by a loop-invariant expression
  - No code between loop starts in `collapse` set
  - Needed to split iteration space (maybe in advance)
- Iterations should be independent
  - Compiler may not stop you if you screw this up!
- Iterations may be assigned out-of-order on one thread!
  - Unless the loop is declared `monotonic`

Reduction loops

How would you parallelize something like this?

```
1       double sum = 0;
2       for (int i = 0; i < N; ++i)
3           sum += x[i];
```

The variables in “sum” must be shared if parallelized.

Reduction loops

OpenMP reduction clause: reduction (op : list)

```
1       double sum = 0, x[MAX];
2       #pragma omp parallel for reduction(+:sum)
3       for (int i = 0; i < N; ++i)
4           sum += x[i];
```

- A local copy of each “sum” variable is made and initialized depending on the “op” (e.g. here “+”).
- Updates occur on the local copy.
- Local copies are reduced into a single value and combined with the original global value.

Exercise: Pi with loops and a reduction

Ordered

OK, what about something like this?

```
1       for (int i = 0; i < N; ++i) {
2           float res = work(i);
3           printf("result for %d is %f\n", i, res);
4       }
```

Work is *mostly* independent, but not entirely.

Ordered

Solution: `ordered` directive in loop with `ordered clause`

```
1       #pragma omp parallel for ordered
2       for (int i = 0; i < N; ++i) {
3           int res = work(i);
4       #pragma ordered
5           printf("result for %d is %f\n", i, res);
6       }
```

Ensures the `ordered` code executes in loop order.
The ordered construct defines an ordered region: the
Statements in ordered region execute in iteration order in the same order as if they were executed on a serial processor.

SIMD loops

```
1       #pragma omp parallel simd reduction(+:sum) aligned(a:64)
2       for (int i = 0; i < N; ++i) {
3           a[i] = b[i] * c[i];
4           sum = sum + a[i];
5       }
```

Can also declare vectorized functions:

```
1       #pragma omp declare simd
2       float myfunc(float a, float b, float c)
3       {
4           return a*b + c;
5       }
```

Other parallel work divisions

- `sections`: like cobegin/coend
- `single`: do only in one thread (e.g. I/O)
- `master`: do only in master thread; others skip

Sections

```
1       #pragma omp parallel
2       {
3           #pragma omp sections nowait
4           {
5               #pragma omp section
6               do_something();
7
8               #pragma omp section
9               do_something();
10
11              #pragma omp section
12              do_something();
13              // No implicit barrier here
14          }
15          // Implicit barrier here
16      }
```

`sections nowait` to kill barrier.

Tasks

Tasks are independent units of work.
Threads are assigned to perform the work of each task.
Task involves:

- Task construct: `task` directive plus structured block
- Task: Task construct + data

Tasks are handled by run time, complete at barriers or `taskwait`.

Example: List traversal

```
1  #pragma omp parallel
2  {
3      #pragma omp single nowait
4      {
5          for (link_t* link = head; link; link = link->next)
6              #pragma omp task firstprivate(link)
7              process(link);
8      }
9      // Implicit barrier
10 }
```

One thread generates tasks, others execute them.

Example: Tree traversal

```
1  int tree_max(node_t* n)
2  {
3      int lmax, rmax;
4      if (n->is_leaf)
5          return n->value;
6
7      #pragma omp task shared(lmax)
8          lmax = tree_max(n->l);
9      #pragma omp task shared(rmax)
10         rmax = tree_max(n->l);
11     #pragma omp taskwait
12
13     return max(lmax, rmax);
14 }
```

The `taskwait` waits for all child tasks.

Task dependencies

What happens if one task produces what another needs?

```
1       #pragma omp task depend(out:x)
2       x = foo();
3       #pragma omp task depend(in:x)
4       y = bar(x);
```

Parallel loops

```
1  /* Compute dot of x and y of length n */
2  int i, tid;
3  double my_dot, dot = 0;
4  #pragma omp parallel shared(dot,x,y,n) private(i,my_dot)
5  {
6    tid = omp_get_thread_num();
7    my_dot = 0;
8
9  #pragma omp for
10   for (i = 0; i < n; ++i)
11     my_dot += x[i]*y[i];
12
13 #pragma omp critical
14   dot += my_dot;
15 }
```

Parallel loops

```
1  /* Compute dot of x and y of length n */
2  int i, tid;
3  double dot = 0;
4  #pragma omp parallel shared(x,y,n) private(i) reduction(+:dot)
5  {
6  #pragma omp for
7    for (i = 0; i < n; ++i)
8      dot += x[i]*y[i];
9  }
```

Parallel loop scheduling

Partition index space different ways:

- `static[(chunk)]`: decide at start of loop; default chunk is `n/nthreads`. Lowest overhead, most potential load imbalance.
- `dynamic[(chunk)]`: each thread takes `chunk` iterations when it has time; default `chunk` is 1. Higher overhead, but automatically balances load.
- `guided`: take chunks of size unassigned iterations/threads; chunks get smaller toward end of loop. Somewhere between `static` and `dynamic`.
- `auto`: up to the system!

Default behavior is implementation-dependent.